

1. Design of clock divider circuit using RTL

(a) Generation of 50MHz clock from 100MHz clock

Aim

To design and verify a clock divider by two circuit using RTL (Verilog HDL) that generates a lower-frequency clock (50MHz) from a high-frequency input clock (100MHz).

Software Required

- Xilinx Vivado Design Suite
- Vivado Simulator
- PC or laptop

Procedure

1. Create a new RTL project in Xilinx Vivado.
2. Add a Verilog module for the clock divider design.
3. Write the RTL code using a counter or toggle logic.
4. Create a testbench to generate clock and reset signals.
5. Run behavioral simulation using Vivado Simulator.
6. Observe the frequency-divided output clock waveform.

Clock divider by two circuit

```
module clock_divider_by2 (  
    input wire clk,  
    input wire rst,  
    output reg clk_out  
);
```

```
always @(posedge clk or posedge rst) begin
```

```
    if (rst)
```

```
        clk_out <= 1'b0;
```

```
    else
```

```
        clk_out <= ~clk_out;
```

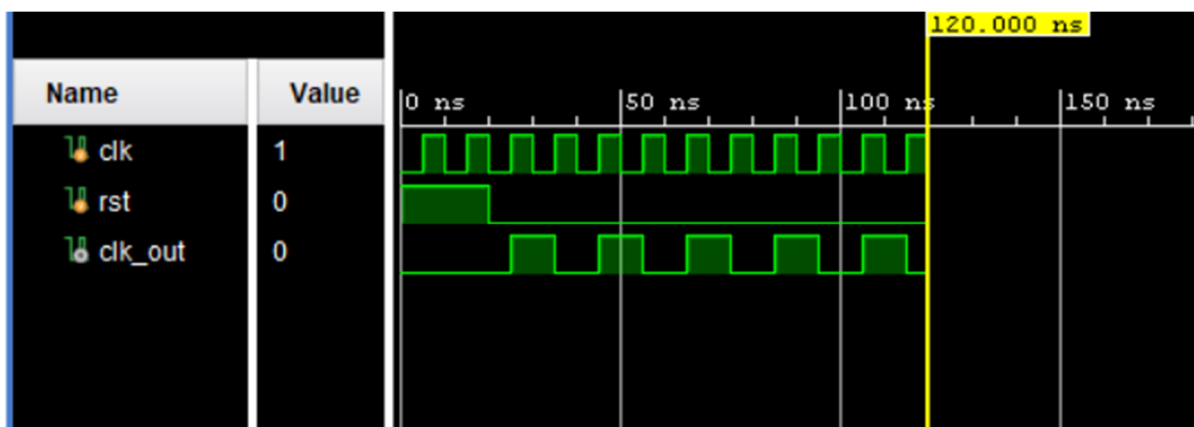
```
end
```

```
endmodule
```

Testbench program

```
`timescale 1ns / 1ps
module tb_clock_divider_by2;
reg clk;
reg rst;
wire clk_out;
clock_divider_by2 uut (
    .clk(clk),
    .rst(rst),
    .clk_out(clk_out)
);
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;
    #20 rst = 0;
    #100;
    $stop;
end
endmodule
```

Output



Result

The clock divider by two circuit was successfully designed using RTL Verilog. Simulation results confirm that the output clock frequency is correctly divided from the input clock as per the specified division factor.

(b) Generation of 25MHz clock from 100MHz clock

Aim

To design and verify a clock divider by four circuit using RTL (Verilog HDL) that generates a lower-frequency clock (25MHz) from a high-frequency input clock (100MHz).

Software Required

- Xilinx Vivado Design Suite
- Vivado Simulator
- PC or laptop

Procedure

1. Create a new RTL project in Xilinx Vivado.
2. Add a Verilog module for the clock divider design.
3. Write the RTL code using a counter or toggle logic.
4. Create a testbench to generate clock and reset signals.
5. Run behavioral simulation using Vivado Simulator.
6. Observe the frequency-divided output clock waveform.

Clock divider by four circuit

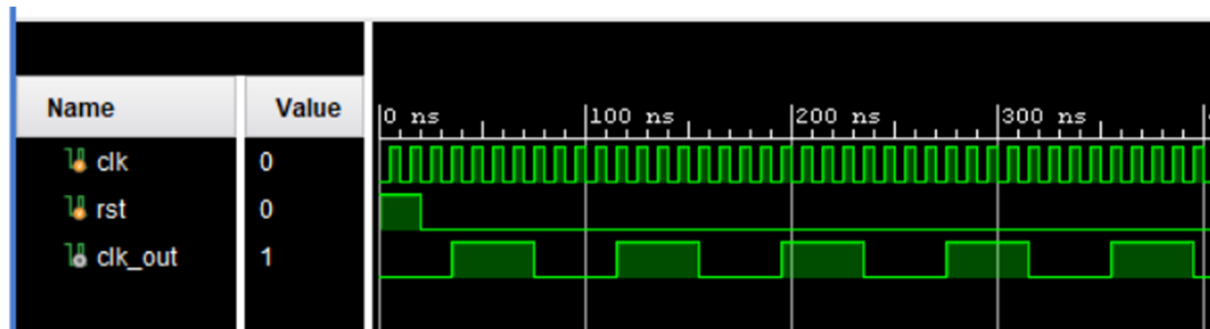
```
module clock_divider_by4 (  
    input wire clk,    // Input clock  
    input wire rst,    // Asynchronous reset  
    output reg clk_out // Divided clock output  
);  
  
reg [1:0] count;  
  
always @(posedge clk or posedge rst) begin  
    if (rst) begin  
        count <= 2'b00;  
        clk_out <= 1'b0;  
    end else begin
```

```
        count <= count + 1;
        if (count == 2'b01)
            clk_out <= ~clk_out;
        end
    end
end
endmodule
```

Testbench program

```
`timescale 1ns / 1ps
module tb_clock_divider_by4;
reg clk;
reg rst;
wire clk_out;
clock_divider_by4 uut (
    .clk(clk),
    .rst(rst),
    .clk_out(clk_out)
);
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;
    #20 rst = 0;
    #1000;
    $stop;
end
endmodule
```

Output



Result

The clock divider by four circuit was successfully designed using RTL Verilog. Simulation results confirm that the output clock frequency is correctly divided from the input clock as per the specified division factor.